Amortized analysis solutions

17-E01 Aggregate table cost

Derivation and check

There are16 insertion writes. Growth from full capacities1,2,4,8 copies15 elements. Total31,average31/16<2. This is a sequence accounting average: push9 copies8 and is expensive,while push16 fills the last slot without resizing.

17-E02 Accounting credit

Derivation and check

Charge3 each time. Operation1 costs1,capacity1,balance2. Operation2 costs2 including one copy,capacity2,balance3. Operation3 costs3 including two copies,capacity4,balance3. Operation4 costs1 with no growth,balance5. Growth occurs on the first insertion beyond old capacity,not when filling its last slot. Total fees dominate costs for every prefix,keeping credit nonnegative. Fees can vary if earlier fixed surpluses still cover growth.

17-E03 Shrink thrashing

Derivation and check

After shrinking size4 to capacity4,the next insertion immediately grows to8 and copies4. Repeated insertion/deletion across4/5 can make each operation Θ(size). Shrink only at quarter load instead:capacity8,size2 shrinks to4,leaving room before growth. This hysteresis supports O(1) amortized cost. The general conclusion still needs the piecewise-potential proof,not merely a few traces.

17-E04 Counter potential with a nonzero initial state

Derivation and check

The first all-ones→zeros increment has c1=b,Φ0=b,Φ1=0,andĉ1=0. A nonoverflow with t trailing ones has c=t+1,ΔΦ=1−t,ĉ=2; overflow hasĉ=0. Telescoping gives$\Sigma c=\Sigma \hat{c}+\Phi 0-\Phi m \leq 2m+b$ because$\Phi m \geq 0$. Zero amortized cost is not zero actual cost; discarding positive initial potential b would incorrectly claim2m from every initial state.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Amortized analysis solutions (continued)

17-E05 Mixed resize potential cases

Derivation and check

Insertion states(n,M,Φ) are(1,1,1),(2,2,2),(3,4,2),(4,4,4),(5,8,2),with c=[1,2,3,1,5],ĉ=[2,3,3,3,3]. Deletion states are(4,8,0),(3,8,1),(2,4,0),(1,2,0),(0,0,0),with c=[1,1,3,2,1],ĉ=[−1,2,2,2,1]. Actual12+8=20 equals amortized14+6=20 since endpoint potentials are0. A negative charge means released potential exceeds this operation’ s cost. For all sequences separately verify no-resize,half-crossing,growth,shrink,and empty cases haveĉ≤3;the lecture proves those generally,while this finite table checks arithmetic.

17-E06 Bit cost of a growing counter

Derivation and check

Bit j flips floor(m/2^j) times. Write m=Σ_k b_k2^k and interchange the finite sums:Σ_jΣ_(k≥j)b_k2^(k−j)=Σ_k b_k(2^(k+1)−1)=2m−popcount(m),strictly below2m for m>0. With decrement,alternate between2^b−1 and2^b. Every crossing flips b+1 bits,so arbitrarily long alternation averagesΘ(b),not a b-independent constant. Monotone increment analysis does not automatically cover reverse operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Amortized analysis solutions (continued)

Verification code appendix: amortized_checks.py

```
"""Exact operation traces for the stated table and counter cost models."""
from fractions import Fraction
from itertools import product


def potential(n, capacity):
    if capacity == 0:
        assert n == 0
        return Fraction(0)
    return Fraction(2*n-capacity) if 2*n>=capacity else Fraction(capacity,2)-n


def step(n, capacity, operation):
    before=potential(n,capacity)
    copied=0
    if operation=="I":
        if capacity==0:
            capacity=1
        elif n==capacity:
            copied=n
            capacity*=2
        n+=1
    else:
        if n==0:
            return None
        n-=1
        if n==0:
            capacity=0
        elif capacity>=4 and 4*n<=capacity:
            copied=n
            capacity//=2
    actual=1+copied
    after=potential(n,capacity)
    return n,capacity,actual,after,actual+after-before


def trace(operations):
    n=capacity=0
    rows=[]
    for operation in operations:
        result=step(n,capacity,operation)
        if result is None:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Amortized analysis solutions (continued)

```
            return None
        n,capacity,cost,phi,amortized=result
        rows.append((operation,n,capacity,cost,phi,amortized))
        assert phi>=0 and amortized<=3
    assert sum(row[3] for row in rows)==sum(row[5] for row in rows)-potential(n,capacity)
    return rows


def main():
    rows=trace("I"*5+"D"*5)
    assert [row[3] for row in rows]==[1,2,3,1,5,1,1,3,2,1]
    assert [row[5] for row in rows]==[2,3,3,3,3,-1,2,2,2,1]
    assert sum(row[3] for row in rows)==20
    for m in range(1,2049):
        total=sum((m>>j) for j in range(m.bit_length()))
        assert total==2*m-m.bit_count()<2*m
    n=capacity=0
    total=0
    for m in range(1,1025):
        n,capacity,cost,_,_=step(n,capacity,"I")
        total+=cost
        assert total<3*m
    assert sum([1,2,4])==7>5

    legal=0
    for length in range(13):
        for ops in product("ID",repeat=length):
            if trace(ops) is not None:
                legal+=1
    for capacity in (4,8,16,32,64,128):
        for n in range(capacity//4+1,capacity+1):
            for operation in "ID":
                result=step(n,capacity,operation)
                assert result and result[3]>=0 and result[4]<=3
    print(f"PASS table_trace_total=20 table_exhaustive_legal_sequences={legal}
counter_exact_totals=2048 insert_prefixes=1024 amortized_bound=3")


if __name__=="__main__":
    main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study